LESSON 05 · CI / CD · 50 MIN

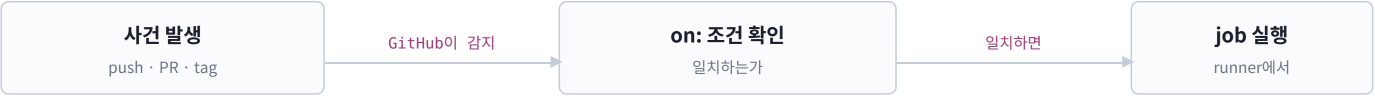
5교시 CI 구축 — push하면 알아서 돌게 만들기

워크플로가 안 도는 이유는 대부분 코드가 아니라 트리거 설정입니다. 최소한의 파일로 시작해서 일부러 깨뜨려 보고, 매트릭스와 캐시를 붙여 실전 형태까지 만듭니다.

개념 12분 · 실습 28분 · 정리와 질문 10분

OVERVIEW

사건에서 실행까지


사건 발생
push · PR · tag

GitHub이 감지

on: 조건 확인
일치하는가

일치하면

job 실행
runner에서

> **on:** 에 적히지 않은 사건은 아무리 push해도 워크플로를 깨우지 못합니다. "Actions 탭이 비어 있어요"의 첫 번째 원인이 이것입니다.

REFERENCE

이 시간에 쓰는 명령

명령	하는 일	언제 쓰나
`on:`	어떤 사건에 반응할지	파일에서 가장 먼저 확인할 곳
`jobs:`	독립적으로 도는 작업 단위	기본은 병렬 실행
`runs-on: ubuntu-latest`	어떤 기계에서 돌릴지	보통 ubuntu 가 가장 빠르고 싸다
`actions/checkout@v4`	내 저장소 코드를 가져온다	거의 모든 워크플로의 첫 스텝
`actions/setup-python@v5`	파이썬을 설치한다	cache: pip 으로 캐시까지
`strategy.matrix`	조합만큼 job 을 복제	여러 버전 동시 검증
`fail-fast`	하나 실패하면 나머지 취소	기본값 true
`actions/upload-artifact@v4`	결과 파일을 남긴다	리포트·로그 보관

FILES

실습용 파일

.github/workflows/test.yml

```yaml
name: test

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip

      - run: pip install -r requirements.txt

      - run: pytest -v
```

HANDS-ON

실습 순서

STEP 01 저장소와 의존성 파일 준비

CI가 돌 수 있는 상태를 만듭니다.

```
# 지금까지 만든 폴더를 git 저장소로
git init && git add . && git commit -m "테스트 추가"

# requirements.txt 를 만듭니다
pytest
pytest-cov
pytest-mock

# GitHub 에 새 저장소를 만들고 연결
git remote add origin https://github.com/아이디/저장소.git
git branch -M main && git push -u origin main
```

확인 포인트

□ GitHub 에 코드가 올라갔다
□ requirements.txt 가 저장소에 포함되어 있다

> **requirements.txt 없이는 CI가 아무것도 설치하지 못합니다.** 내 컴퓨터에 깔려 있는 것은 runner 와 무관합니다.

STEP 02 첫 워크플로 파일 만들기

가장 작은 CI를 구성합니다.

```
mkdir -p .github/workflows
# 위 '실습용 파일'의 test.yml 을 .github/workflows/ 에 만듭니다
git add . && git commit -m "CI 추가" && git push
```

확인 포인트

□ 경로가 정확히 .github/workflows/test.yml 이다
□ 들여쓰기가 전부 스페이스다 (탭 금지)

> 폴더 이름 앞의 **점(.)**을 빠뜨리면 GitHub이 인식하지 못합니다. 파일 확장자는 .yml 과 .yaml 둘 다 됩니다.

STEP 03 자동 실행 확인하고 로그 읽기

실제로 도는 것을 눈으로 봅니다.

```
# GitHub 저장소 → Actions 탭
# 방금 push 한 커밋의 워크플로를 클릭
# test job → 각 step 을 펼쳐 봅니다
```

확인 포인트

□ Set up job → checkout → setup-python → install → pytest 순서로 진행된다
□ pytest step 에 내 컴퓨터에서 본 것과 같은 출력이 나온다
□ 초록 체크가 뜬다

> 각 step 의 **소요 시간**을 봐 두세요. 다음 단계에서 캐시를 붙이면 install 시간이 어떻게 줄어드는지 비교할 수 있습니다.

STEP 04 일부러 깨뜨려 보기 — 오늘의 핵심 (1)

빨간 X 를 만들고 원인을 찾아봅니다.

```
# 테스트 하나의 기대값을 틀리게 바꿉니다
assert cart.total() == 1
git commit -am "일부러 실패" && git push

# Actions 에서 실패한 실행을 열어 봅니다
```

확인 포인트

□ 빨간 X 가 뜬다
□ 실패한 step 이 자동으로 펼쳐진다
□ 3교시에서 배운 실패 리포트가 그대로 보인다

> **CI 로그를 읽는 법은 로컬과 똑같습니다.** 실패한 step 을 찾고, 그 안의 리포트를 읽으면 됩니다. 확인 후 되돌려 초록불로 만들어 두세요.

STEP 05 트리거를 바꿔 실험하기

on: 설정과 실제 실행의 관계를 확인합니다.

```
# 실험 A · 브랜치를 만들어 push
git switch -c feature/test && git commit --allow-empty -m "실험" && git push -u origin feature/test
# → Actions 에 아무것도 안 뜬다 (branches: [main] 이므로)

# 실험 B · PR 을 열어 본다
# GitHub 에서 Compare & pull request
# → pull_request 트리거로 실행된다

# 실험 C · on: 에 브랜치 추가
  push:
    branches: [main, 'feature/**']
```

확인 포인트

□ A: 브랜치 push 는 무시된다
□ B: PR 을 열면 실행된다
□ C: 설정 후에는 브랜치 push 에도 실행된다

PR 체크가 초록일 때만 병합되게 하려면 **Settings → Branches → Branch protection rule** 에서 이 워크플로를 required check 로 지정하세요. 이때부터 CI가 실제로 방어선 역할을 합니다.

STEP 06 matrix 로 여러 버전 검증 — 오늘의 핵심 (2)

조합만큼 job 이 복제되는 것을 봅니다.

```
jobs:
  test:
    runs-on: ${{ matrix.os }}
    strategy:
      fail-fast: true
      matrix:
        os: [ubuntu-latest, windows-latest]
        python-version: ["3.11", "3.12"]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}
          cache: pip
      - run: pip install -r requirements.txt
      - run: pytest -v
```

확인 포인트

□ job 이 4개(2 × 2)로 늘어난다
□ 각각 test (ubuntu-latest, 3.11) 처럼 이름이 붙는다
□ 동시에 실행된다

조합은 **곱셈**이라 금세 불어납니다. 4개면 사용 시간도 4배입니다.

fail-fast 를 false 로 바꿔 다시 실행해 보세요. 하나가 실패해도 나머지가 끝까지 돌아 "특정 버전만 문제인지, 전부 문제인지"를 한 번에 알 수 있습니다.

STEP 07 캐시 효과 확인하기

설치 시간이 줄어드는 것을 숫자로 봅니다.

```
# setup-python 의 cache: pip 은 이미 넣어 두었습니다
# 같은 커밋을 다시 실행해 비교합니다
# Actions → 실행 → 우측 상단 Re-run all jobs
```

확인 포인트

□ 첫 실행: pip install 이 수십 초 걸린다
□ 두 번째: Cache restored 메시지와 함께 크게 줄어든다
□ requirements.txt 를 고치면 캐시 키가 바뀌어 다시 받는다

캐시 키는 **requirements.txt 내용**으로 만들어집니다. 파일이 그대로면 재사용하고, 한 줄이라도 바뀌면 새로 받습니다. Docker 레이어 캐시와 정확히 같은 원리입니다.

STEP 08 리포트를 아티팩트로 남기기

실행이 끝나도 결과를 볼 수 있게 합니다.

```
      - run: pytest --cov=src --cov-report=html --junitxml=report.xml

      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: coverage-${{ matrix.os }}-${{ matrix.python-version }}
          path: htmlcov/
```

확인 포인트

□ 실행 결과 페이지 아래 Artifacts 에서 내려받을 수 있다
□ 압축을 풀고 index.html 을 열면 커버리지 리포트가 보인다

> **if: always()** 가 중요합니다. 이게 없으면 테스트가 실패했을 때 리포트가 업로드되지 않습니다 — 정작 리포트가 가장 필요한 순간에 말이죠. 이름에 matrix 값을 넣은 것은 조합마다 이름이 겹치면 안 되기 때문입니다.

STEP 09 배지 붙이고 정리

저장소 첫 화면에서 상태가 보이게 합니다.

```
# README.md 맨 위에 추가
![test](https://github.com/아이디/저장소/actions/workflows/test.yml/badge.svg)

git commit -am "배지 추가" && git push
```

확인 포인트

□ README 에 초록색 passing 배지가 보인다
□ 실패하면 빨간 failing 으로 바뀐다

> 배지는 장식이 아니라 **지금 main 이 건강한가**를 한눈에 보여 주는 신호등입니다.

TROUBLESHOOTING

자주 만나는 오류

증상	원인	해결
Actions 탭에 아무것도 안 뜸	경로·이름 오류 또는 on: 불일치	.github/workflows/*.yml 경로 확인 · on: 의 브랜치 확인
ModuleNotFoundError (로컬은 되는데 CI만)	requirements.txt 누락	필요한 패키지를 전부 requirements.txt 에 기록
yaml 파싱 오류	탭 사용 또는 들여쓰기 불일치	스페이스 2칸으로 통일 · 편집기에서 탭 표시 켜기
Resource not accessible by integration	GITHUB_TOKEN 권한 부족	워크플로에 permissions: 를 명시 · 6교시에서 다룹니다
캐시가 매번 새로 받아짐	requirements.txt 가 계속 바뀜 또는 경로 문제	파일이 저장소 루트에 있는지 확인

REVIEW

복습 체크리스트 — 답할 수 있으면 통과

□ on: 에 pull_request 만 있으면 main 에 직접 push 했을 때 실행되나요?
□ actions/checkout 을 빼면 어떻게 되나요?
□ matrix 에 os 3개, python 3개면 job 은 몇 개가 되나요?
□ fail-fast: false 로 두면 무엇이 좋아지고 무엇을 잃나요?
□ 테스트가 실패해도 리포트를 남기려면 무엇이 필요한가요?

한 줄 정리

CI는 사람이 잊어도 기계가 기억하게 만드는 장치입니다. 초록불이 병합의 조건이 될 때부터 진짜 효과가 납니다.

다음 시간 → 6교시 — 통과한 것만 자동으로 내보내기

WORKSHEET

막힌 STEP 번호 · 화면에 나온 오류 메시지 · 물어볼 것